# Artificial Intelligence Software, Lecture 4

## Data versioning by ClearML, Labeling tools

MIPT

Center for Cognitive Modeling

# The Philosophy: Data ≠ Code

- **Non-linear Progress:** Dataset development is often non-linear, making it difficult to find the exact commit associated with a specific data version in a traditional Git tree.
- **State vs. Logic:** Data represents the state used for training, while code represents the logic; treating them identically leads to a loss of experimental transparency.
- **Volume and Scale:** Machine learning involves massive amounts of data that versioning systems like Git were never designed to handle.

# Why Git Fails for Large Data

- **Repository Bloat:** Committing large binary files (GBs or TBs) into Git leads to bloated repositories and painfully slow operations like cloning.
- **Inefficient Diffing:** Git is optimized for text-based diffs; for binary data, any minor change requires storing a full new copy, leading to storage nightmares.
- **Lack of ML Context:** Standard Git cannot track experiment-specific metrics, model weights, or dataset ancestry (lineage).

# Benefits of Separate Storage (S3/Object Storage)

- **Accessibility:** Storing data in object storage like AWS S3 or Yandex Object Storage ensures it can be accessed from any machine (local, cloud, or remote agent).
- **Single Source of Truth:** Centralized storage provides a consistent view for the entire team, preventing conflicts where members work on different dataset copies.
- **Scalability:** Cloud-native storage offers virtually infinite capacity and supports high-bandwidth data transfers.

# Modern Infrastructure: S3 and Parquet

- **S3 (Object Storage):**
  - A universal protocol for object storage (Amazon AWS, Yandex Cloud, etc.) where every file is accessible via a unique URL.
  - Data is organized into **Buckets** (logical containers), serving as a centralized ”Single Source of Truth” for the entire team.
  - Highly scalable and cost-effective for storing massive datasets and model artifacts.
- **Parquet (Columnar Format):**
  - A binary, **column-oriented** storage format designed for efficient Big Data processing.
  - **Efficiency:** Significantly faster for read operations than CSV or JSON because it allows reading only required columns.
  - **Metadata-aware:** Stores schema information and statistics internally, eliminating the need for slow schema inference.

# Reproducibility and Data Lineage

- **The Reproducibility Bedrock:** Reproducibility requires the exact combination of code, environment, and data; without versioned data, debugging is impossible.
- **Task Linking:** MLOps tools (ClearML, DVC) link specific dataset versions to training tasks, so it is always clear which data produced which model.
- **Ancestry Tracking:** Dedicated data modules track where specific parts of data came from and how they evolved over time.

# Sources and Further Reading

- ClearML Data Management - DeepSchool
- DVC - The Git of AI: Why Every ML Engineer Needs It
- Data Versioning: Best Practices for ML Engineers
- Tutorial: Data and Model Versioning - DVC Documentation

# Where to Find Public Datasets

Finding high-quality data is the first step in any ML project. Leading platforms include:

- **Kaggle**, **Hugging Face**.
- **Roboflow Universe**, **Supervisly**.
- **Academic & Open Data: Papers with Code** links datasets to research.

# Common Data Modalities

Modern ML tasks handle diverse data types across various domains:

- **Vision:** Images (Classification/Detection), Video (Tracking), 3D Point Clouds/LiDAR, and Medical Imagery (DICOM).
- **Natural Language (NLP):** Raw Text (Corpora), Audio (Speech-to-Text), and Document-based data.
- **Structured Data:** Tabular data (CSV/Parquet), Time-series, and Geospatial data.
- **Specialized:** LiDAR for autonomous driving and multispectral imagery for spectral-aware evaluation.

# References and Resources

- Hugging Face Datasets Hub
- Kaggle Public Datasets
- Roboflow Universe: Computer Vision Data
- ClearML Data Management Guide
- DVC Command Reference: get
- Top 10 Public Dataset Finders — SuperAnnotate

# Understanding Labels and Annotations

- **What is Data Annotation?** The process of adding metadata to images, video, or text to help ML algorithms understand the content.
- **Labels:** The specific ”tags” or class names assigned to objects (e.g., ”Car”, ”Pedestrian”, ”Tumor”).
- **Annotation Modalities:**
  - **Bounding Box:** 2D rectangles for object detection.
  - **Polygon/Mask:** Precise pixel-level shapes for instance or semantic segmentation.
  - **Keypoints:** Dots marking specific coordinates (e.g., human pose estimation).
  - **Cuboids:** 3D boxes projected onto 2D images or 3D point clouds.
  - **Tags:** Classification labels for the entire image or scene.

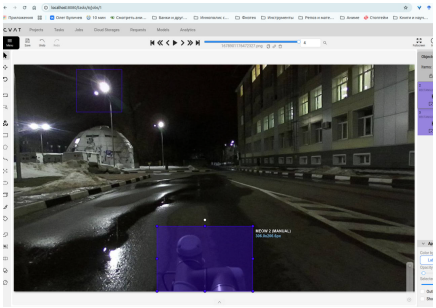
# The Era of Pre-annotation: SAM

- **What is SAM?** The **Segment Anything Model** (developed by Meta) is a foundation model for image segmentation.
- **Pre-annotation Logic:** Instead of drawing pixels manually, an engineer clicks an object, and SAM generates a high-quality **mask** instantly.
- **Efficiency Boost:** Tools like Supervisely and CVAT integrate SAM as ”Smart Tools,” allowing for ”one-click” labels that drastically reduce manual labor.
- **Active Learning:** Pre-trained models suggest annotations which humans then correct, accelerating the dataset creation cycle.

# CVAT (Computer Vision Annotation Tool)

**Core Features:**

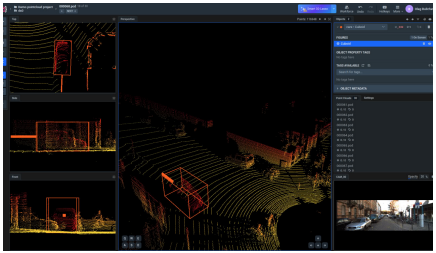
- **Open Source:** Highly customizable and supports self-hosting.
- **Versatility:** Supports 2D images, video interpolation (tracking), and 3D LiDAR.
- **Formats:** Compatible with 19+ formats including YOLO, COCO, and Pascal VOC.
- **Team Roles:** Granular access for managers and annotators.



# Supervisely

**Core Features:**

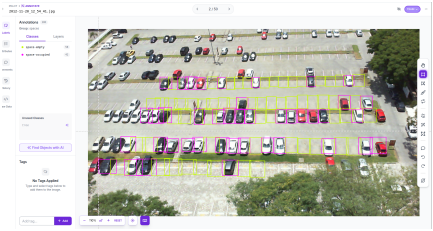
- **Platform OS:** Comprehensive ecosystem for the entire CV lifecycle (Label → Train → Deploy).
- **App Ecosystem:** Hundreds of specialized apps for data cleaning, visualization, and model conversion.
- **Sensor Fusion:** Real-time synchronization between 2D images and 3D LiDAR point clouds.
- **Enterprise Scale:** Designed for heavy high-resolution imagery and complex ontologies.



# Roboflow

**Core Features:**

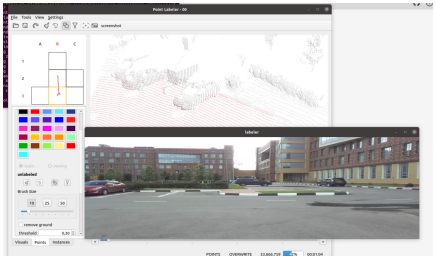
- **Community Hub:** Access to the ”Roboflow Universe” with 1M+ open-source datasets.
- **Web-First:** Simple, intuitive browser interface optimized for rapid prototyping.
- **Preprocessing:** Integrated tools for resizing, augmentations, and health checks.
- **Deployment:** One-click deployment to various edge devices via Roboflow Train.



# Point_labeler

**Core Features:**

- **LiDAR Focus:** Specifically designed for labeling millions of points in 3D scans.
- **Tile-based Rendering:** Loads scan tiles to handle massive KITTI point clouds efficiently.
- **OpenGL Shaders:** Uses modern graphics processing to render complex 3D scenes in real-time.
- **SemanticKITTI:** The primary tool used to create the landmark SemanticKITTI dataset.



# References


- CVAT: Leading Data Annotation Platform
- Supervisely: Computer Vision Platform
- Roboflow: Computer Vision Datasets & Models
- GitHub: point_labeler tool
- CVAT Full Guide — Data Light

# The Landscape of Data Versioning

Modern MLOps relies on three primary tools for managing datasets:

- **DVC (Data Version Control):** Acts as a Git extension, storing lightweight .dvc pointers in Git while physical data resides in remote storage like S3.
- **Hugging Face Datasets:** A specialized library for NLP, Vision, and Audio, integrated with the HF Hub for sharing and versioning.
- **ClearML Data:** A module of the ClearML platform that manages data independently of code, adhering to the ”Data is not Code” philosophy.

# ClearML Data: Structure and Philosophy

- **Philosophy:** ClearML believes data progress is often non-linear and should not be stored in a Git tree.
- **Everything is a Task:** In the ClearML ecosystem, datasets, experiments, and pipelines are all handled as the same object type: a **Task**.
- **Decoupling:** Data management is handled via the `clearml-data` CLI or the `clearml.Dataset` Python SDK.
- **Single Source of Truth:** The ClearML Server tracks experiment runs and dataset versions over time for exact recreation.

# Key Concepts of ClearML Data

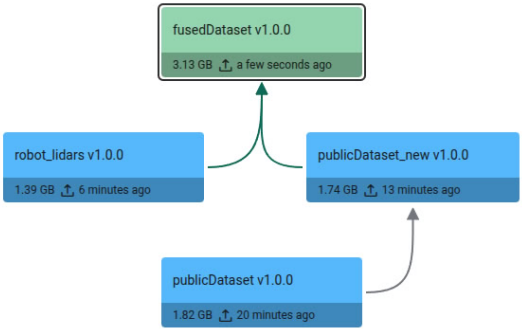
- **Accessibility:** Ensures data is accessible from any machine (local, cloud, or remote agents).
- **Ancestry/Lineage:** Automatically tracks where specific parts of data came from through parent-child relationships.
- **Differentiable Storage:** Only uploads the **delta** (differences) between a new version and its parent, saving bandwidth and space.
- **Immutability:** Once a dataset version is ”Finalized,” it is locked to ensure reproducibility.

# Dataset Management via CLI

The clearml-data utility allows for quick operations:

- **Creation:** clearml-data create --project "Proj" --name "DS".
- **Adding Files:** clearml-data add --path ./local_folder (recursively adds all files).
- **Finalizing:** clearml-data close (uploads files and locks the version as ”Final”).
- **Synchronization:** clearml-data sync --folder ./data (combines create, add, and upload in one step).

# ClearML Dataset Lineage



# Reproducibility: Linking Data to Tasks

- **Automatic Connection:** When a training task uses `Dataset.get()`, ClearML automatically records the Dataset ID in the task's configuration.
- **Traceability:** Users can see exactly which version of a dataset produced which model in the Web UI.
- **Collaboration:** Team members can use `get_local_copy()` to work with identical data slices, ensuring consistent results across the team.

# References


- ClearML Data Management — DeepSchool
- ClearML Data Interface Documentation
- Hugging Face Datasets Documentation
- DVC: Versioning Data and Models
- ClearML Official YouTube: Getting Started Series

# Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521